programiz.com/c-programming/online-compiler/

Programiz
C Online Compiler

Programiz PRO >

main.c

Share Run

```c
#include <stdio.h>
#include <ctype.h>

#define MAX 100

int stack[MAX];
int top = -1;

// Push operation
void push(int value)
{
    top++;
    stack[top] = value;
}

// Pop operation
int pop()
{
    return stack[top--];
}

int main()
{
    char postfix[MAX];
    int i, op1, op2, result;

```

Output

Clear

```
Enter postfix expression: 25*4/8
Result = 8

=== Code Execution Successful ===
```

Compiler - Programiz

programiz.com/c-programming/online-compiler/

gramiz

ne Compiler

ain.c

Share Run

```c
    char postfix[MAX];
    int i, op1, op2, result;

    printf("Enter postfix expression: ");
    scanf("%s", postfix);

    for(i = 0; postfix[i] != '\0'; i++)
    {
        // If character is operand
        if(isdigit(postfix[i]))
        {
            push(postfix[i] - '0');
        }
        // If character is operator
        else
        {
            op2 = pop();
            op1 = pop();

            switch(postfix[i])
            {
                case '+':
                    result = op1 + op2;
                    break;

                case '-':
```

Output

Clear

```
Enter postfix expression: 25*4/8
Result = 8

=== Code Execution Successful ===
```

ompiler - Programiz

programiz.com/c-programming/online-compiler/

ramiz

ne Compiler

in.c

Share Run

```c
            case '*':
                result = op1 * op2;
                break;

            case '/':
                result = op1 / op2;
                break;

            case '%':
                result = op1 % op2;
                break;

            default:
                printf("Invalid operator");
                return 1;
        }

        push(result);
    }
}

printf("Result = %d", pop());

return 0;
}
```

Output

Clear

```
Enter postfix expression: 25*4/8
Result = 8

=== Code Execution Successful ===
```

ompiler - Programiz

programiz.com/c-programming/online-compiler/

ramiz

ne Compiler

in.c

Share Run

```c
            case '*':
                result = op1 * op2;
                break;

            case '/':
                result = op1 / op2;
                break;

            case '%':
                result = op1 % op2;
                break;

            default:
                printf("Invalid operator");
                return 1;
        }

        push(result);
    }
}

printf("Result = %d", pop());

return 0;
}
```

Output

Clear

```
Enter postfix expression: 25*4/8
Result = 8

=== Code Execution Successful ===
```

```c
#include <stdio.h>
#define MAX 5

int queue[MAX];
int front = -1, rear = -1;

// Insert element
void enqueue(int value)
{
    if(rear == MAX - 1)
    {
        printf("Queue Overflow\n");
    }
    else
    {
        if(front == -1)
            front = 0;

        rear++;
        queue[rear] = value;

        printf("%d inserted\n", value);
    }
}
```

```
10 inserted
20 inserted
30 inserted
Queue Elements:
10 20 30
10 deleted
Queue Elements:
20 30


=== Code Execution Successful ===
```

```c
// Delete element
void dequeue()
{
    if(front == -1 || front > rear)
    {
        printf("Queue Underflow\n");
    }
    else
    {
        printf("%d deleted\n", queue[front]);
        front++;
    }
}

// Display queue
void display()
{
    int i;

    if(front == -1 || front > rear)
```

```
10 inserted
20 inserted
30 inserted
Queue Elements:
10 20 30
10 deleted
Queue Elements:
20 30


=== Code Execution Successful ===
```

```c

        for(i = front; i <= rear; i++)
        {
            printf("%d ", queue[i]);
        }
    }

    printf("\n");
}

int main()
{
    enqueue(10);
    enqueue(20);
    enqueue(30);

    display();

    dequeue();

    display();

    return 0;
}
```

```
20 inserted
30 inserted
Queue Elements:
10 20 30
10 deleted
Queue Elements:
20 30


=== Code Execution Successful ===
```